Steward — Product Requirements Document

Oct 17, 2026 · @nikolaj

Overview

Steward is an AI office manager that turns a photo of a problem into a diagnosed, ticketed and costed fix, so office managers stop being their office's help desk.

Someone posts a photo in Slack or on the handheld device. Steward sees it, matches it to the office's own equipment, decides whether staff can fix it or a supplier must be called, speaks the steps in a voice that suits the severity, and logs the outcome so it improves with use. It ships as a web app, a Slack integration and a handheld multi-tool, the Steward Bot, which can also dock onto a small rover base.

Pitch in one line: Steward turns "someone take a look at this" into a diagnosed, ticketed, costed and remembered fix.

Status of this document: draft v0.1 for the hackathon build. Facts come from the team's notes and the working prototype. Anything not yet decided is listed under Open questions, and anything we have assumed is labelled as such.

Problem

Office managers spend their day routing small problems, which leaves little time for work that improves the office.

A coffee machine breaks, the Wi-Fi drops, a fuse trips, a lunch order has to go out. Each issue is low value, but urgent to whoever is affected, so people interrupt the office manager. The office manager then works out what is wrong, finds the right supplier, chases the invoice and tries to remember what happened last time. None of that knowledge is written down; it lives in one person's head.

Why current tools fail:

- **Ticketing and facilities tools** assume an IT team and a clean, structured request. Office problems arrive as a blurry photo and a message in Slack.
- **General-purpose assistants** (including big-platform ones) start from scratch every time. They do not know the office's equipment, suppliers or history, so they cannot decide what to do next.
- **Nothing is tailored** to the office manager's job, so the work stays manual, interrupt-driven and invisible to the people who hold the budget.

What it costs:

- Office managers are pulled away from higher-value work by frequent interruptions.
- Key employees such as engineers or PMs are pulled into low value tasks (fixing machines, wifi etc.)
- Budget holders see call-out and invoice costs but not the cause, so repeat faults and wrong bills go unnoticed.
- Nobody can answer a simple question: is the office getting better at handling problems, or worse?

We have not yet measured how much time this takes in a real office. Baseline figures are an open question (see Risks and open questions).

Users and personas

The office manager is the primary user; the other three groups touch Steward because of what the office manager does with it.

Persona	Who they are	What they need	What Steward gives them
Office manager (primary)	The person who keeps the office running and is interrupted all day	Fewer interruptions, faster fixes, less chasing, more autonomy	Triage, diagnosis, escalation and a record of everything, with a handheld device they can carry
COO / budget holder	Owns the facilities and supplier budget	Visibility of cost and trend, confidence that bills are right	Spend and time-to-fix trends, invoice checks, an answer to "are we getting better?"
Third-party stakeholder	A supplier called out to fix something (electrician, IT support, appliance repair, plant care)	A clear brief before they arrive	An escalation with the photo, diagnosis, location and what has already been tried
Regular staff	Anyone in the office who hits a problem and gets pulled into fixing it	A fast way to report, and simple steps when they can fix it themselves	Report by photo in Slack, spoken or written steps, and a notification when it is fixed

Primary scenario: a staff member posts a photo of the kitchen fuse board in Slack. Steward identifies the tripped circuit, tells them how to reset it safely, merges the two other reports into one ticket, and tells the office manager only if it cannot be resolved.

Goals, non-goals and success metrics

The goal is to save the company time by letting office managers resolve more issues themselves, with fewer interruptions.

Goals

1. Turn a photo or a spoken description of a problem into a diagnosis and next step in one interaction.
2. Decide correctly between "staff can fix it" and "call a supplier", and escalate with full context.
3. Merge duplicate reports so one problem is one ticket.

4. Build a per-office memory (equipment, contacts, history, invoices) that makes every later decision better.
5. Give budget holders a trend they can trust: are we getting better as an office?
6. Work in the places people already are: Slack first, then a dedicated handheld device.

Non-goals (for the hackathon and v1)

- Replacing IT service desks or full facilities-management suites.
- Making repairs that need a licensed professional. Steward guides safe first steps and escalates the rest.
- Executing payments. Steward checks and flags invoices; a person approves them.
- Acting autonomously on safety-critical issues without a person in the loop.

Success metrics

Targets are set after a one-week baseline in a real office; the direction is fixed now.

Metric	Direction	How it is measured
Median time to fix	Down	Ticket created to ticket closed
Issues fixed without a supplier call-out	Up	Share of closed tickets with no escalation
Staff interruptions per week	Down	Direct messages to the office manager about issues
Supplier call-out spend	Down	Invoiced call-out cost per month
Duplicate reports merged	Up	Reports attached to an existing ticket / all reports
Wrong or unexpected charges caught	Up	Flagged invoices confirmed as errors
Office manager time saved	Up	Time per ticket before vs after, from a diary study

Hackathon success criteria: a live demo of the full loop (photo, vision, decision, spoken fix, ticket, merge, escalation, invoice check) on the eight scenarios below.

Core scenarios

These eight real office tasks define the minimum Steward must handle end to end; each is built into the prototype's demo data.

Scenario	What Steward sees	Decision	Outcome
Fuse has gone	Photo of the fuse board with a tripped switch	**Critical.** Staff can reset it safely; escalate to the electrician if it trips again with nothing plugged in	Spoken, calm step-by-step reset; duplicate reports merged; reporters notified when fixed
Wi-Fi is broken	Report from several people in the same area, plus network status	**High.** Power-cycle the affected access point; escalate to managed IT with logs if that fails	One ticket for all reporters; supplier receives logs and what was tried
Coffee machine is broken	Photo of the error screen ("grinder jammed")	**Normal.** Staff can fix: empty the hopper to two-thirds, clear the burr	Steps with estimated time; known-issue note saved to memory
Printer is not working	Photo of the display ("toner low")	**Normal.** Staff can fix: swap the spare cartridge, resume the queue	Steps; leased printer, so no call-out cost
Dishwasher is broken	Photo of error code E24	**Normal.** Staff can fix: clear the drain filter	Steps; ticket closed in minutes
Office plants are dying	Photo of yellowing leaves, dry soil	**Low.** Water today; book overdue plant care	Visit booked with the plant-care supplier
Screens need to be installed	Delivery note and meeting-room list	**Low.** Supplier job, not a fault; book the installer and block the rooms	Install booked, calendar updated, rooms blocked
Just Eat delivery needs to go out	Slack RSVPs and dietary notes	**Normal.** Draft the order from last time, adjusted for headcount and allergies	Draft basket for the office manager to approve

Every scenario ends the same way: the outcome is written to the company context so later decisions use it.

Functional requirements

The P0 requirements are the hackathon loop; P1 follows once it works; P2 is later. Change a priority from its dropdown.

Intake and diagnosis

ID	Requirement	Priority
R1	Accept a photo or video posted in the office Slack channel / email and treat it as a report	P0 Must
R2	Accept a photo, video, text or voice report in the web app	P0 Must
R3	Accept a report from the Steward Bot handheld	P1 Should
R4	Accept text-only and voice-only reports with no media	P1 Should
R5	Identify the item in a photo and match it to the office inventory	P0 Must
R6	Read error screens, labels and model plates in the image	P0 Must
R7	Return a diagnosis, a confidence level and fix steps	P0 Must
R8	Ask one follow-up question when confidence is low	P1 Should

Decisions

ID	Requirement	Priority
R9	Classify severity as critical, high, normal or low	P0 Must
R10	Decide between "staff can fix it" and "call a supplier"	P0 Must
R11	Detect duplicate reports of the same issue, merge them into one ticket and keep every reporter	P0 Must
R12	Pick the right supplier and show the call-out cost before escalating	P0 Must
R12a	Classify problem type	P1 Should

Tickets and tasks

ID	Requirement	Priority
R13	Create a ticket holding the photo, diagnosis, steps, item, location and support contact	P0 Must
R14	Let the office manager mark a ticket urgent, change its priority and reorder the list by hand	P0 Must
R15	Mark a ticket fixed and notify the channel and all reporters	P0 Must
R16	Escalate with one action, composing the brief for the supplier automatically	P0 Must

Voice

ID	Requirement	Priority
R17	Speak the diagnosis and steps, with a voice chosen by severity (calmer for critical)	P0 Must
R18	Let the user stop and replay the spoken steps	P1 Should

Company context and memory

ID	Requirement	Priority
R19	Keep an inventory with models, locations, service history and support contacts	P0 Must
R20	Sync the employee directory from the HR system and route issues to people who are in	P1 Should
R21	Remember recurring faults and what fixed them, and use that in later diagnoses	P1 Should

Invoices, insights and extras

ID	Requirement	Priority
R22	Match supplier invoices to tickets from the accounting system	P2 Later
R23	Flag unexpected or duplicate charges and draft a dispute	P2 Later
R24	Require a person to approve any invoice or dispute before it is sent	P2 Later
R25	Show time to fix, self-fix rate, interruptions and spend over time	P1 Should
R26	Suggest actions from patterns, such as replacing a repeatedly failing access point	P2 Later
R27	Draft the team lunch order from Slack RSVPs and dietary notes	P2 Later
R28	Show a 3D office map with pins for each item, coloured by open issues	P2 Later

Company context database

Each office has one context database, and the vision model and the decision model both read from it on every request. This is Steward's moat: it is specific to the office and grows with use.

Area	What it holds	How Steward uses it	Source
Office description	Floors, rooms, layout, house rules, where equipment sits	Locates what is in a photo; places pins on the 3D map	Set up once; updated by the office manager
Employees	Who works here, roles, who is in today, who is on leave, starters and leavers	Routes an issue to someone who is actually in; prepares onboarding tasks	HR system sync (for example HiBob)
Escalation contacts	Suppliers, phone numbers, call-out costs, contract terms, warranties	Chooses who to call and shows the cost first	Entered by the office manager; extended from invoices

Area	What it holds	How Steward uses it	Source
Memory	What Steward has learned: recurring faults, quirks of each item, what worked	Improves diagnoses; drives suggestions such as "replace this access point"	Written back after every ticket
History of tasks	Every ticket, its photos, steps, reporters, timings and outcome	Detects duplicates and repeats; feeds the Insights trend	Created automatically from tickets
Invoices and products	The equipment inventory (models, install dates, service history) and supplier bills	Diagnoses from a photo; checks bills against what actually happened	Accounting system sync (for example Xero); scan-to-add for new items

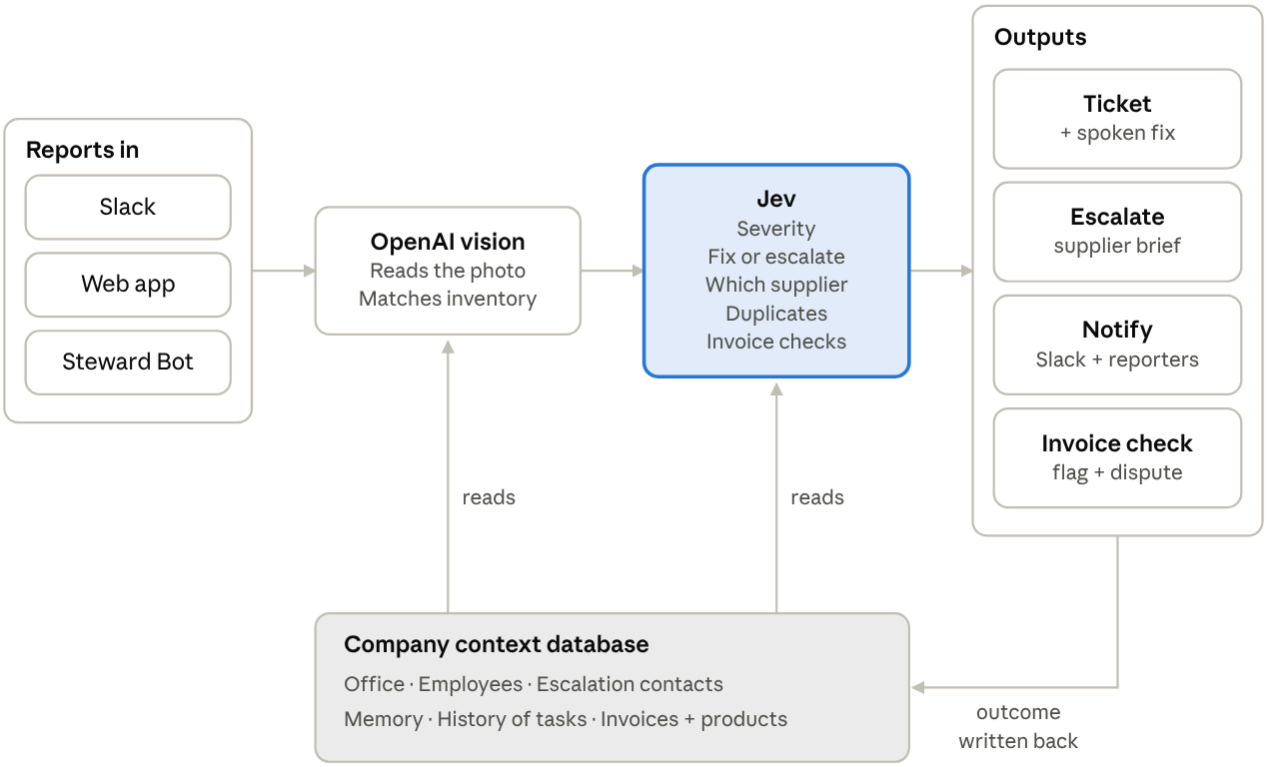
Write-back rule: every resolved ticket updates history, memory and, if a supplier was involved, the invoice match. Nothing Steward learns is left only in a chat.

Scope for the hackathon: all six areas exist as mock data. Memory is the thinnest: the prototype shows hard-coded suggestions rather than real learning.

AI architecture

A report passes through vision, then Jev, and both read the company context; the outcome is written back so the next decision is better.

Vision sees, Jev decides, and both read the company context



architecture · report to vision to Jev to outputs, with the company context feeding both

Vision turns the photo into facts, Jev turns those facts plus the context into a decision, and the app, Slack and the voice carry it out.

Stage	Component	Input	Output
1. Intake	Slack, web app, Steward Bot	Photo, video, text or voice	One report with reporter, time and place
2. See	OpenAI vision	The image plus the relevant inventory	Item match, readings from screens and labels, a diagnosis and a confidence
3. Decide	Jev	The vision result plus the company context	Severity, fix or escalate, supplier, duplicate-of, invoice verdict
4. Act	App, Slack, ElevenLabs	The decision	Ticket, spoken steps, supplier brief, notifications
5. Learn	Company context database	The outcome	Updated history, memory and invoice match

Voice follows severity. Critical uses a calm, steady voice, high is clear and focused, normal is friendly, and low is light and casual. Every spoken step is also shown as text.

Escalation logic. If the decision is "fix", Steward gives the steps and checks back; if it is "escalate", Steward drafts the brief for the supplier from the context (item, location, photo, diagnosis, what was tried, call-out cost) and waits for a person to send it.

Decision contract (proposed). Jev's real interface is an open question; this is the shape the app is built to expect, so the mock can be swapped for the real model.

```
{
  "severity": "critical | high | normal | low",
  "route": "self_fix | escalate",
  "supplier_id": "string or null",
  "duplicate_of": "ticket id or null",
  "confidence": 0.0,
  "reasoning": "one or two sentences shown to the user",
  "needs_approval": true
}
```

Steward Bot device

The Steward Bot is a pocket multi-tool that lets the office manager carry Steward to the problem instead of waiting at a desk. It combines the Rabbit R1's screen, wheel and push-to-talk with a Swiss Army knife's stacked, folding tools.

Form factor (design targets, not engineered figures): an anodised-aluminium slab with stainless tools, about 66 × 112 × 19 mm, with a 2.8-inch portrait touch screen on the front and a tool spine on the back.

Part	Design	Why
Camera barrel	Rotating barrel at the top with two fisheye lenses back to back (360° view); a tilt control rotates it	Sees under desks, into cupboards and down a comms rack
Screen	2.8-inch portrait touch display with a card-stack interface	Shows the ticket, the checklist and the escalate button
Scroll wheel	Thumb wheel on the right edge	Move through tasks one-handed
Push-to-talk	Amber key on the right edge, with four mics and a speaker	Talk to Steward in a loud office; voice chosen by severity
Connectivity	Wi-Fi, SIM or eSIM, USB-C	Works on and off the office network

Part	Design	Why
Tool spine	Six folding tools, three at each end on a shared pivot, under a smoked cover plate	Fix common faults without a call-out
Rover dock	The device clips onto a small wheeled base with a lidar puck	Lets Steward move around and check on issues (later phase)

Tools and the faults they map to

Tool	Fixes	Scenario
Voltage probe	Finds the dead circuit before anyone touches the board	Fuse has gone
Bit driver	Screws and brackets	Screens need to be installed
Pry tool	Panels, kick-plates, ceiling tiles	Screens, dishwasher access
Jam grabber	Pulls out jammed paper	Printer is not working
Soil probe	Reads soil moisture	Office plants are dying
Brush and pick	Clears the grinder burr	Coffee machine is broken

Wi-Fi faults are handled by a built-in network scan, and lunch orders by voice, so neither needs a tool. The scan is a proposal, not yet specified.

Prototype: an interactive 3D model (orbit, fan out the tools, exploded view, rover dock) lives in the app at the device page, and exports an STL for CAD. It is a concept model with no tolerance work, materials or electronics packaging. Moving to real hardware needs an industrial-design and engineering pass.

Integrations

Steward works inside tools the office already uses, so setup is connecting accounts rather than changing habits. Slack is the front door; the rest feed the company context or carry out a decision.

Integration	Purpose	Direction	Prototype status
Slack	Receive photos and messages from a channel; reply in the thread; post "fixed" and tag reporters	In and out	Simulated in the UI
Invoicing / accounting	Match supplier bills to tickets, flag wrong charges, draft disputes	In (bills), out (dispute drafts)	Mock panel; Xero shown as a placeholder
HR software	Employees, who is in or on leave, starters and leavers	In	Mock panel; HiBob shown as a placeholder
Food ordering	Draft the team lunch from RSVPs and dietary notes	Out (draft for approval)	Mock draft
Calendar	Book supplier visits, block rooms during installs	Out	Mock
OpenAI vision	Read photos and videos	Out (images), in (structured result)	Not connected; keyword-matched mock
Jev	Decide severity, self-fix or escalate, supplier, duplicates, invoice checks	Out (context), in (decision)	Not connected; hard-coded mock logic
ElevenLabs	Speak steps with a voice per severity	Out (text), in (audio)	Browser speech as a stand-in
Microsoft Teams	The same flow for offices not on Slack	In and out	Listed, not built

Rules for every integration

1. Read access is the default; any action that spends money, books a supplier or contacts one on the office's behalf asks a person to approve first.
2. Each connection can be switched off from the Connectors screen without losing existing tickets.

3. The vendor for invoicing and HR is the customer's choice; Steward needs a connector per system, so the first two are an open question.

Non-functional requirements

Steward handles photos of a workplace and decisions about money and safety, so privacy and human oversight come before speed.

Privacy and data

- Each office's context database is isolated; no data is shared across offices.
- The product is personalised to the office's own devices and data, and that data is private to the customer.
- Photos can capture people. Steward stores the minimum needed for a ticket, supports deleting a ticket and its media, and does not use one customer's data to serve another.
- Employee data comes from the HR system on a read-only, least-access basis.

Safety

- Steward gives guidance only for steps a non-specialist can do safely, such as resetting a tripped circuit with nothing plugged in.
- Anything electrical beyond that, or any situation Steward is unsure about, escalates to a qualified supplier and says so plainly.
- Critical issues use the calmer voice profile and short, ordered steps.

Human oversight

- A person approves anything that spends money, books a supplier or sends a dispute.
- Every decision shows its reasoning, so the office manager can see why Steward chose to fix or escalate and can override it.

Performance and reliability (proposed targets for the demo)

- A spoken diagnosis within about 10 seconds of a photo being posted.
- If a connected service is down, Steward still creates the ticket and tells the user what it could not do.
- Voice falls back to on-screen text when audio is unavailable.

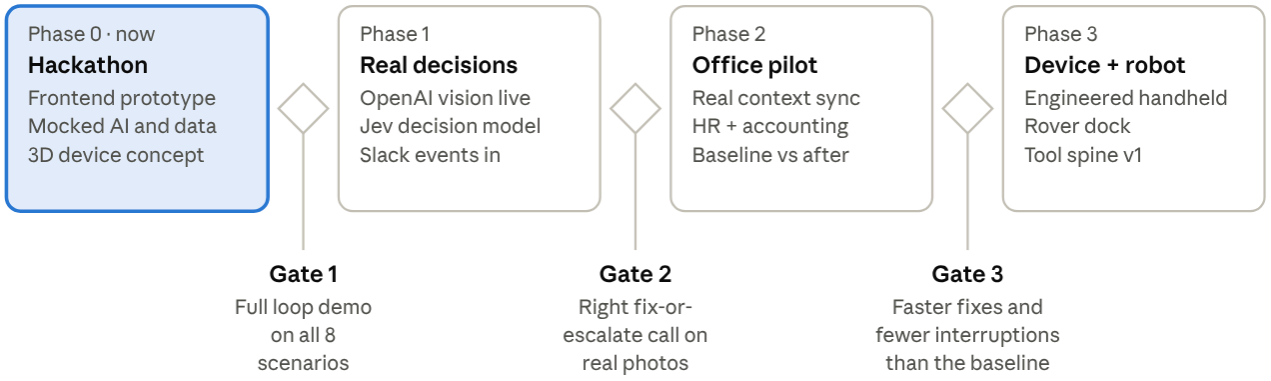
Accessibility and usability

- Every spoken instruction is also shown as text.
- The app works on a phone, since office managers are on their feet.

Roadmap and prototype status

The hackathon prototype covers the whole experience with mocked intelligence; the next phase connects the real vision and decision models.

The prototype is phase 0 of 4; each gate unlocks the next phase



Diamonds are gates: criteria to pass before the next phase. Criteria are proposed; dates are not set.

roadmap · 4 phases, 3 gates

Each phase starts only when the gate before it passes. Dates are not set; they depend on the first pilot customer.

What the prototype contains today

Area	In the app	Real or mocked
Tasks	Board with photos, urgent tag, duplicate counts, manual reordering, mark fixed and notify	Mock data
Ask Steward	Photo, video, text or mic report; pipeline view; duplicate check; spoken fix	Keyword-matched mock; browser speech
Agents	A conversation per job with Steward and specialist agents (Facilities, IT, Lunch)	Mock replies; not yet defined in this PRD
Insights	"Are we getting better?" metrics, charts and suggestions	Illustrative numbers
Office	3D office map with pins, and the inventory as an expandable list	Mock floor plan and items

Area	In the app	Real or mocked
Connectors	Slack, invoicing, HR, ordering, calendar, AI and voice, with toggles and an invoice-checking panel	Mock sync and invoices
Steward Bot	Interactive 3D concept with the tool spine, exploded view, rover dock and STL export	Concept model only
OpenAI vision, Jev, ElevenLabs, Slack events	Not connected	Not built

Risks, assumptions and open questions

The biggest risks are that the decision layer is not yet connected and that no real-office baseline exists, so the headline claim (time saved) is unproven.

Risks

Risk	Why it matters	Mitigation
Vision misreads a photo or picks the wrong item	A wrong diagnosis wastes time or is unsafe	Show confidence, ask a follow-up when low, always allow "escalate"
Unsafe advice on electrical or similar faults	Harm to people and liability	Limit guidance to safe first steps; escalate the rest; log every instruction
Decision model not yet integrated	The core promise (the right call) is mocked today	Define the decision contract and test it on the eight scenarios first
Context database stays empty or stale	Diagnoses and routing degrade	Scan-to-add for items; sync from accounting and HR; prompt for missing details
Hardware is a large leap from a concept model	Cost, supply chain and certification are unknown	Ship the software first; treat the device as a later phase and a pitch asset
Photos of people and workplaces raise privacy concerns	Customer trust and compliance	Minimal storage, deletion, per-office isolation

Risk	Why it matters	Mitigation
Big platforms add office features	Competitive pressure	Depend on the office-specific context and memory they do not have

Assumptions (not yet validated)

- Office managers will report and fix through Slack and a phone before they would adopt a new device.
- Most daily issues are repeats that a good inventory and history can diagnose.
- Suppliers will accept a structured brief from Steward in place of a phone call.

Open questions

- [ ] What exactly does Jev decide, what is its input and output, and how is it called? This PRD assumes severity, self-fix or escalate, supplier choice, duplicates and invoice checks.
- [ ] Which invoicing system and which HR system do we connect to first? The prototype uses Xero and HiBob as placeholders.
- [ ] What is the real baseline: how many issues a week, and how much of the office manager's time do they take?
- [ ] Who is the first customer, and what size of office is the target?
- [ ] How is Steward priced, and who pays: the office, the COO or the landlord?
- [ ] Which supplier relationships, if any, are real for the demo?
- [ ] Does the handheld device ship in the first year, or stay a concept for the pitch?
- [ ] How do the specialist agents in the app (Facilities, IT, Lunch) relate to Steward and to Jev: separate models, or one decision model with different context?